



**WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	3
Opis pracy	4
Zakres tematyczny	6
Ujęte w zakresie	6
Poza zakresem	6
Artefakty pracy	7
Organizacja pracy	7
Podstawy teoretyczne	8
Wprowadzenie do sieci konwolucyjnych (CNN)	8
Operacja splotu	8
Receptywne pole	9
Nieliniowości i normalizacja	9
Pooling i część klasyfikacyjna	10
Regularizacja i trening	10
Triki architektoniczne	10
Ekwiwariancja translacyjna (kontrast do rotacyjnej)	10
Inwariancja translacyjna i rotacyjna	11
Linear-polar vs. log-polar	11
Problemy z rotacyjną inwariancją w klasycznych CNN	11
Przegląd literatury (E(2)-equivariant, CyCNN)	12
Opis zbiorów danych	13
MNIST (cyfry odręczne)	13
GTSRB Gray (znaki drogowe w odcienach szarości)	13
GTSRB RGB (znaki drogowe w kolorze (RGB))	13
LEGO (obiekty 3d rzutowane na 2d)	13
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	13
Architektury modeli	14
VGG - wariant E (VGG-19, „3×3 everywhere”)	14
ResNet-56 (CIFAR, 6n+2 z n=9)	14
Wersje cykliczne: CyVGG-E i CyResNet-56	15
Uzgodnienia I/O i selektor modeli	15
Standardowe CNN	15
Rotacyjnie inwariantne sieci (CyResNet, CyVGG)	15
Transformacje polarne: linearpolar vs logpolar	16
Implementacja i środowisko eksperymentalne	16
Szczegóły implementacyjne: warstwa CyConv2d (CUDA)	16
Python, PyTorch	16
Struktura projektu	16
Automatyzacja: skrypty trenowania, testowania, ewaluacji	16

Obsługa GPU, Docker, WSL	16
Organizacja logów, modeli, confusion matrixów	16
Eksperymenty	17
Scenariusze trenowania/testowania (opis JSON)	17
Pomiar skuteczności (accuracy, macierze pomyłek)	17
Śledzenie metryk: średnia, mediana, odchylenie standardowe	17
Analiza skuteczności względem rotacji	17
Ranking modeli	17
Porównanie wyników	17
CyCNN vs klasyczne CNN	17
VGG vs CyVGG	17
Resnet vs CyResNet	17
CyVGG vs CyResNet	17
Wpływ transformacji (linearpolar vs logpolar)	17
Wydajność na różnych zbiorach	17
Wnioski	17
Skuteczność rotacyjnych architektur	17
Wnioski z automatyzacji i systematyzacji ewaluacji	17
Propozycje dalszych badań	17
Aneks	17
Listingi kodów	17
Dodatkowe wykresy, tablice wyników	17
Bibliografia	18

Wstęp

Obrazy otaczają nas z każdej strony: od zdjęć ze smartfonów, zdjęcia satelitarne, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Rdzeń trudności to brak naturalnej inwariantności względem rotacji: standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami [1], [2].

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedną to poszerzanie danych o zrotowane przykłady - poprawia odporność, ale wydłuża trening i nie gwarantuje uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant) [3], [4], sieci cykliczne (CyCNN; w szczególności **CyVGG** i **CyResNet**) operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny: by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręcznie napisane cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu** [5], mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach: **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów; środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy** i **jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, jak wpływają na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmienniczości względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniałyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności rozwiązań zaproponowanych w literaturze, ukierunkowanych na odporność modeli na rotację danych wejściowych (np. CyCNN [4]).

W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaki drogowe (zarówno w kolorze, jak i w odcieniach szarości) oraz obiekty 3D rzutowane na 2D (klocki LEGO), rozszerzone o kontrolowane rotacje obrazów.

Na podstawie tych zbiorów została przeprowadzona implementacja i ewaluacja wybranych architektur rotacyjnie inwariantnych z wykorzystaniem **PyTorch**. Obliczenia zostały wykonane przy użyciu kart graficznych **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co umożliwiło przyspieszenie trenowania. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwolucyjnych w celu oceny realnych korzyści z rozwiązań inwariantnych względem rotacji.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- **Język programowania Python** - podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki wszechstronności i ekosystemowi bibliotek [6] Środowiska były izolowane dzięki użyciu `venv`.

Frameworki uczenia maszynowego:

- **PyTorch** - elastyczny framework do budowy, trenowania i wdrażania modeli ML/DL (w tym własnych warstw, jak `CyConv`) [7].
- **Modele cykliczne (CyCNN)**. W pracy zostało przyjęte podejście, w którym obraz został przemapowany do współrzędnych (ρ, φ) . Dzięki temu obrót R_α staje się przesunięciem o α po osi φ . Konwolucje zostały zastąpione warstwami cylindrycznymi (`CyConv`) z cyklicznym paddingiem po φ . Dla każdego filtra zostało przygotowanych n orientacji; odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia powoduje cykliczny shift po tej osi, a pooling po orientacjach daje inwariancję względem rotacji. Został ustawiony stały środek układu polarnych, stała siatka próbkowania oraz biliniarna interpolacja; padding po φ został ustawiony na cykliczny. Implementacja została wykonana w **PyTorchu** (punkt odniesienia: `CyCNN` [4]).
- **Wykorzystanie akceleracji GPU (NVIDIA)** - obliczenia zostały znacząco przyspieszone dzięki użyciu kart **RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**. Frameworki takie jak wspierają **PyTorch CUDA**, **Tensor** oraz **cuDNN**, co umożliwia efektywne wykorzystanie zasobów GPU [8], [9]. Monitorowanie i diagnostyka zostały wykonane z użyciem narzędzia `nvidia-smi`.
-
- **Tensor Cores (Ampere)**. Zastosowane karty RTX (3070 Ti, 3060) mają rdzenie Tensor, które sprzętowo przyspieszają operacje macierzowe (konwolucje/matmul). Biblioteki **cuDNN/cuBLAS** na architekturze **Ampere** domyślnie mogą używać trybu **TF32** dla obciążeń FP32, co daje dodatkowe przyspieszenie bez zmian w modelu. Dodatkowo, w **PyTorch** możliwe jest włączenie **mieszanej precyzji** (FP16/BF16) przez **AMP** w miejscach, gdzie to bezpieczne, przy włączeniu tego feature, zwykle przyspiesza to trening przy porównywalnej jakości (szczegóły w dokumentacji). [10], [11], [12], [13]
- **System operacyjny: Linux (Ubuntu LTS)**. Główne środowisko uruchomieniowe stanowił system operacyjny **Ubuntu** (dystrybucja LTS) posiadający stabilne jądro, pakiety z APT, ła-

twą integrację ze sterownikami NVIDIA i CUDA. Treningi uruchamiane były **lokalnie** na maszynach z GPU NVIDIA. [14]. Dla zgodności ze środowiskami Windows używano też wariantu **WSL2** (ten sam obraz Dockera i ta sama konfiguracja) [15].

- **Konteneryzacja za pomocą Dockera** - odizolowane środowiska uruchomieniowe ułatwiły replikację i współdzielenie projektu, także z obsługą GPU [16].

Zakres tematyczny

Niniejsza praca dotyczy odporności modeli klasyfikacji obrazów na rotacje w płaszczyźnie 2D. Skupiono się na porównaniu klasycznych architektur z ich wersjami rotacyjnie inwariantnymi oraz na wpływie przekształceń polarnych na jakość predykcji. Badania zostały przeprowadzone na obrazach 2D i rotacjach planarnych. W szczególności zostały porównane warianty bazowe **VGG/ResNet** z wersjami cyklicznymi **CyVGG/CyResNet**, a także wpływ mapowania **linear-polar** i **log-polar**. Eksperymenty zostały wykonane na zbiorach **MNIST**, **GTSRB_gray**, **GTSRB_RGB** i **LEGO** z kontrolowanymi rotacjami.

Ujęte w zakresie

- **Architektury modeli:** zostały zaimplementowane i porównane warianty bazowe **VGG** oraz **ResNet** [17], [18], a także ich wersje cykliczne **CyVGG** i **CyResNet** (modele rotacyjnie inwariantne) [4].
- **Przekształcenia polarne:** została oceniona użyteczność mapowania **linear-polar** oraz **log-polar** jako etapów wstępnego przetwarzania służących „prostowaniu” rotacji do przesunięć [4], [19].
- **Zbiory danych:** zostały wykorzystane następujące zbiory: **MNIST** (odręczne cyfry, 28x28 → 32x32, grayscale) [20], **LEGO** (syntetyczne obiekty 3D rzutowane na 2D, 96x96, grayscale), **GTSRB** (znaki drogowe, 32x32, grayscale) oraz **GTSRB_RGB** (wersja kolorowa) [21]. Zbiory zostały rozszerzone o kontrolowane rotacje oraz zostały przygotowane spójne podziały zbiorów na train/val/test.
- **Augmentacja i protokół:** zostały zdefiniowane zakresy kątów, liczba powtórzeń i podział na zbiory train/val/test (uczący/ walidacyjny/testowy), z możliwością powtórzenia trenowań.
- **Środowisko i implementacja:** został wykorzystany **PyTorch** z akceleracją **CUDA/cuDNN** na kartach **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **NVIDIA GeForce RTX 3060 12 GB** [7], [8], [9]. Przygotowane zostały skrypty w Pythonie do trenowania, testowania i ewaluacji [6]. Środowisko uruchomieniowe zostało ustandaryzowane z użyciem **Dockera** [16].
- **Metryki i analiza:** została przeprowadzona ocena jakości (accuracy, macierze pomyłek), analiza stabilności (średnia/mediana/odchylenie standardowe), wpływ kąta rotacji na skuteczność oraz koszt obliczeniowy (czas trenowania, rozmiar modelu).

Poza zakresem

- Detekcja obiektów i segmentacja - w pracy rozpatrywana jest wyłącznie klasyfikacja [22], [23].
- Inwariancja względem skali, ścinania i pełnych przekształceń afinicznych - analizowana jest tylko rotacja w płaszczyźnie [24], [25].
- Rotacje w geometrii 3D oraz zagadnienia widzenia stereo - pozostają poza zakresem [26], [27].

- Trening na bardzo dużych korpusach z pre-treningiem self-supervised oraz szerokim AutoML/hyper-search - nie został realizowany [28], [29], [30].
- Odporność na silne zakłócenia (szum, okluzje) - poza zakresem; skupiono się wyłącznie na rotacji [31], [32].

Artefakty pracy

- Repozytoria z kodem, skryptami i plikami konfiguracyjnymi (PyTorch).
- Pliki konfiguracyjne eksperymentów i opis danych.
- Wytrenowane wagi modeli (wybrane checkpointy) oraz raporty z ewaluacji.
- Tekst pracy z dokumentacją eksperymentów i wnioskami.

Organizacja pracy

Struktura pracy została ułożona tak, by od podstaw przejść do wyników. W rozdziale **Podstawy teoretyczne** zostały zebrane pojęcia i narzędzia: CNN, inwariancja/ekwawariancja, przekształcenia polarne oraz prace pokrewne (G-CNN, E(2)-equivariant, CyCNN). W **Opisie zbiorów danych** zostały przedstawione MNIST, LEGO, **GTSRB_gray** i **GTSRB_RGB** oraz sposób augmentacji (rotacje, podziały train/val/test). W **Architekturach modeli** zostały opisane warianty bazowe **VGG/ResNet** oraz wersje cykliczne **CyVGG/CyResNet**, wraz z transformacjami linear-polar / log-polar. Rozdział **Implementacja i środowisko** zawiera szczegóły techniczne: **PyTorch**, **CUDA** i **cuDNN** (RTX 3070 Ti 8 GB, RTX 3060 12 GB), **Docker**, strukturę projektu i skrypty. W **Eksperymentach** zostały zdefiniowane scenariusze, metryki i sposób ewaluacji. Dalej, w **Porównaniu wyników**, zostały zestawione modele (VGG vs. CyVGG, ResNet vs. CyResNet, wpływ transformacji) i omówiona stabilność/czas. Na końcu sekcja **Wnioski** zbierają najważniejsze **obserwacje** i wskazuje kierunki dalszych badań; **Aneks** zawiera kody i dodatkowe wykresy.

Podstawy teoretyczne

Wprowadzenie do sieci konwolucyjnych (CNN)

Sieci konwolucyjne (CNN) zostały zaprojektowane do pracy na danych o strukturze siatkowej lub macierzowej, takich jak obrazy dwuwymiarowe (2D). Ich kluczowe cechy to **lokalne receptywne pola**, **współdzielone wagi** oraz **operacja splotu**. Dzięki temu możliwe jest skalowanie modeli na duże obrazy oraz lepsze uogólnianie niż w przypadku sieci posiadającej pełne połączenia.

Zamiast analizować cały obraz jednocześnie, CNN wykorzystują mały filtr, który przesuwa się po lokalnych fragmentach obrazów. W ten sposób uczą się prostych detektorów (np. krawędzi, tekstur, kształtów), a w wyższych warstwach - bardziej złożonych cech [1], [33].

Dla przesunięcia $\mathcal{T}_t$ oraz jądra K zachodzi własność **ekwiwariancji translacyjnej**:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K).$$

Intuicyjnie oznacza to, że jeśli obraz zostanie przesunięty, to odpowiadająca mu mapa cech również przesunie się o ten sam wektor.

Inwariancja na przesunięcia osiągana jest w praktyce poprzez pooling (lokalny lub globalny) bądź zwiększenie kroku (stride). Rzadsze próbkowanie powoduje, że wynik klasyfikacji nie zależy od dokładnej pozycji obiektu [1], [2].

Operacja splotu

Intuicyjnie ten sam mały filtr „przesuwa się” po obrazie i oblicza ważoną sumę pikseli. Dzięki temu uzyskuje się **współdzielenie wag** (liczba parametrów nie rośnie wraz z H, W) oraz **lokalność obliczeń** [1], [2].

Kształty.

Wejście: $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$.

Zestaw jąder: $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$.

Wyjście: $Y \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$.

Definicja (pojedynczy kanał wyjściowy c):

$$Y_c(u, v) = \sum_{i=1}^{C_{\text{in}}} \sum_{a,b} K_{c,i}(a, b) X_i(u-a, v-b).$$

W praktyce większość frameworków oblicza **korelację krzyżową** (bez odwracania jądra), mimo że w API funkcja nazywana jest conv [2].

Nie ma to jednak znaczenia dla procesu uczenia - sieć i tak dobierze właściwe wagi.

Stride, padding, rozmiary Parametry „geometrii” warstwy:

- **padding** p - ile pikseli dodajemy na brzegach;
- **stride** s - co ile pikseli przesuwamy okno;
- **dylacja** d - „rozciga” jądro poprzez wstawienie przerw między próbkami.

Uwaga o „same/valid/stride” a ekwiwariancji

Dokładna ekwiwariancja translacyjna zachodzi przy splocie bez zmiany rozmiaru. W praktyce **padding „same”**, **stride** > 1 i **pooling** wprowadzają drobne odchylenia (aliasing na siatce próbkowania), co obniża „idealność” ekwiwariancji - efekt ten jest znany i opisywany w literaturze [2], [34].

Rozmiar wyjścia (dla jądra $k \times k$):

$$H' = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

Typowe ustawienia.

- *valid*: $p = 0$ - mapy cech maleją;
- *same* (dla $s = 1$): $p = \lfloor k/2 \rfloor$ - $H' = H$, $W' = W$;
- *stride* > 1: wbudowane **podpróbkowanie** (mniej obliczeń, mniejsza rozdzielczość);
- *dylacja* > 1: większe **efektywne** pole widzenia bez nowych parametrów (częste w detekcji/segmentacji) [2].

Receptywne pole

Receptywne pole to fragment obrazu „widziany” przez daną aktywację w mapie cech. Im głębsza warstwa, tym pole jest większe, ponieważ kolejne sploty agregują informacje z coraz większych fragmentów wejścia. Dla jąder k_ℓ i kroków s_ℓ zachodzi:

$$R_1 = k_1, \quad R_\ell = R_{\ell-1} + (k_\ell - 1) \prod_{j<\ell} s_j.$$

W praktyce nie wszystkie piksele w obrębie tego pola mają taki sam wpływ. Największy wpływ ma obszar centralny, a znaczenie maleje ku brzegom - rozkład wpływu przypomina rozkład Gaussa. Z tego powodu w bazowych architekturach VGG i ResNet dobiera się głębokość tak, aby przy stosowanej rozdzielczości objąć cały obiekt bez utraty istotnego kontekstu [35].

W wariantach **CyCNN** po przejściu do współrzędnych (ρ, φ) oraz przy **cyklicznym paddingu** wzdłuż φ sieć „widzi” pełny zakres orientacji bez artefaktów brzegowych, co ułatwia stabilne uczenie cech niezależnych od kąta [4].

Receptywne pole w układzie polarnym

Po mapowaniu $(x, y) \rightarrow (\rho, \varphi)$ receptywne pole staje się „wąskim paskiem” wzdłuż promienia ρ i stabilnym po kącie φ . Dzięki temu obrót $\mathcal{R}_\alpha$ na wejściu jest równoważny **przesunięciu** o α po osi φ . Warstwy typu CyConv z **cyklicznym paddingiem** wzdłuż φ nie „tną” informacji na brzegach, co wzmacnia ekwiwariancję rotacyjną [4].

Nieliniowości i normalizacja

Blok konwolucyjny pozostaje **taki sam** we wszystkich wariantach (bazowych i cyklicznych). Celem porównania jest wpływ **rotacji**, a nie dobór aktywacji.

Normalizację zastosowano standardowo, aby stabilizować uczenie. W wariantach **CyCNN** statystyki normalizacji liczone są **wspólnie po osi orientacji**, tak aby **nie faworyzować żadnego kąta** i nie naruszać własności rotacyjnych [4], [36].

Pooling i część klasyfikacyjna

W modelach **CyCNN** inwariancja względem rotacji uzyskiwana jest przez **agregację po orientacjach** (pooling po osi kątów). Następnie, we wszystkich modelach, stosowany jest **global average pooling (GAP)** oraz pojedyncza warstwa liniowa w klasyfikatorze. GAP redukuje liczbę parametrów i zmniejsza zależność od położenia w obrębie map cech [37]. Część klasyfikacyjna pozostaje taka sama w wariantach bazowych i cyklicznych, aby izolować wpływ części „rotacyjnej” [4].

Pooling po orientacjach - szczegóły praktyczne

Agregacja po osi *orientacja* (avg lub max) realizuje **inwariancję rotacyjną**. Na wynik wpływa liczba orientacji **n**: większe **n** oznacza dokładniejszą rozdzielczość kątową (mniejszy błąd zaokrąglenia $2\pi/n$), ale też wyższy koszt obliczeń i pamięci. W eksperymentach utrzymano identyczny klasyfikator za poolingiem, aby jednoznacznie mierzyć wpływ części „rotacyjnej” [4].

Regularizacja i trening

Protokół trenowania został **zamrożony** między wariantami (te same: liczba epok, rozmiar batcha, budżet obliczeń, wczesne zatrzymanie - wszystko zgodnie z planem eksperymentu). Augmentacje ograniczono do tych **niezależnych od rotacji** w testach „czysto architektonicznych”. Augmentację rotacją stosowano wyłącznie w eksperymentach kontrolnych - aby pokazać różnicę między „augmentacją” a „architekturą”.

Triki architektoniczne

- **Bazy:** VGG-19 (bloki 3×3) i ResNet-56 (wariant CIFAR, bloki 3×3).
- **Wersje cykliczne:** podmiana $\text{Conv} \rightarrow \text{CyConv}$, dodanie osi **orientacja** i **cykliczny padding** po φ ; pozostałe elementy (głębokość, liczba kanałów) dobrano tak, aby utrzymać *porównywalny budżet parametrów/FLOPs* względem baz.
- **Bez innych trików:** nie wprowadzano zmian niezwiązanych z rotacją, aby nie mieszać efektów [4].

Ekwiwariancja translacyjna (kontrast do rotacyjnej)

Dla przesunięcia $\mathcal{T}_t$ i splotu zachodzi:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K),$$

co tłumaczy, dlaczego klasyczne CNN dobrze radzą sobie z translacją [2]. Brak analogicznego mechanizmu dla rotacji motywuje użycie przekształceń polarnych i/lub modeli cyklicznych w dalszej części.

Ekwiwariancja rotacyjna w dyskretnej grupie C_n

Dla indeksu orientacji $k \in \{0, \dots, n-1\}$ i kąta $\theta_k = \frac{2\pi k}{n}$ ekwiwariancję można zapisać jako

$$\Phi(\mathcal{R}_{\theta_k} X)[\cdot, m] = \Phi(X)[\cdot, m+k \bmod n],$$

czyli **obrót wejścia = cykliczne przesunięcie po osi orientacji**. Następnie pooling po tej osi znosi zależność od kąta (inwariancja) [4].

Ekwiwariancja (przewidywalna zmiana reprezentacji):

$$\Phi(\mathcal{T}_t X) = \Pi_t \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Pi_\alpha \Phi(X),$$

gdzie Π to działanie grupy na cechach (dla translacji - przesunięcie map, dla rotacji - np. **cykliczny shift** po osi „orientacja”) [1], [38].

Inwariancja translacyjna i rotacyjna

Niech $\mathcal{T}_t$ oznacza przesunięcie o wektor t , $\mathcal{R}_\alpha$ - obrót o kąt α , a Φ - mapę cech / model.

Inwariancja (wynik nie zależy od transformacji):

$$\Phi(\mathcal{T}_t X) = \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Phi(X).$$

W praktyce:

- **translacja**: uśrednianie / podpróbkowanie (GAP, stride);
- **rotacja**: (a) augmentacja o obroty, (b) architektura ze śledzeniem orientacji (oś „kąt” + pooling po orientacjach), (c) mapowanie do współrzędnych polarnych, gdzie obrót staje się przesunięciem po osi φ [2], [4], [19].

Linear-polar vs. log-polar

- **Linear-polar**: równy przyrost w pikselach po ρ i φ . Stabilne kąty, prosta implementacja; dobre pod **inwariancję rotacyjną**.
- **Log-polar**: skala rośnie logarytmicznie po ρ - zmiany skali stają się przesunięciami po osi promienia. Jest to idealne rozwiązanie pod **rotacje i skale**, ale bliżej środka rośnie gęstość próbkowania i wrażliwość na wybór środka [19].

W praktyce stosowana jest interpolacja biliniarna + **cykliczny padding po φ** , stały środek. Blisko $\rho=0$ warto wygładzić / wykluczyć kilka próbek, aby uniknąć „osobliwości” środka [4].

Problemy z rotacyjną inwariancją w klasycznych CNN

- **Kierunkowość filtrów**. Pojedynczy kernel jest wrażliwy głównie na jedną orientację - bez dodatkowych mechanizmów sieć „gubi” obroty.
- **Augmentacja nie domyka całości**. Rotacje pomagają, ale wydłużają trening i zostawiają „dziury” między kątami (przy małym kroku i ograniczonym budżecie).
- **Aliasing / interpolacja**. Obracanie dyskretnych obrazów wprowadza artefakty i szum [34].

- **Krawędzie i padding.** „same/zero” łamie symetrię przy brzegach - odpowiedzi nie są idealnie ekwiwariantne.
- **Brak osi orientacji.** Standardowe CNN nie przechowują informacji, „pod jakim kątem” wykryto aktywację - trudno to później zwinąć do rozpoznawania niezależnego od kąta.

Przegląd literatury (E(2)-equivariant, CyCNN)

CyCNN (podejście użyte w pracy). Obraz przemapowano do (ρ, φ) i przetwarzano warstwami cylindrycznymi (CyConv) z **cyklicznym paddingiem** wzdłuż φ . Dla każdego filtra wykorzystano n orientacji (grupa C_n). Obrót wejścia z C_n wywołuje **cykliczny shift** po osi orientacji (**ekwiwariancja**), a **pooling po orientacjach** daje **inwariancję**. W badaniach użyto CyVGG i CyResNet [4].

E(2)-equivariant / steerable CNNs (kontekst). Sploty grupowe i jądra sterowalne umożliwiają ekwiwariancję względem translacji i rotacji (również dla ciągłych kątów) w grupie E(2). Wymaga to projektowania jąder zgodnie z reprezentacjami grupy i zwykle wiąże się z większym kosztem obliczeń. W tej pracy traktujemy temat ten jako tło teoretyczne [3], [39], [40].

Opis zbiorów danych

MNIST (cyfry odręczne)

GTSRB Gray (znaki drogowe w odcieniach szarości)

GTSRB RGB (znaki drogowe w kolorze (RGB))

LEGO (obiekty 3d rzutowane na 2d)

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Architektury modeli

(VGG-E, ResNet-56, CyCNN)

Przegląd. Wykorzystane są bazy **VGG** (wariant **E**) i **ResNet** (wariant **56**) w ustawieniu „cifar” (obrazy 32×32). Konwolucje to głównie 3×3 z padding=1; w VGG pojawia się okresowy MaxPool2d(2), a w ResNet zmniejszanie rozdzielczości odbywa się przez stride=2. Po części konwolucyjnej stosowany jest **global average pooling (GAP)** oraz **klasyfikator**. W VGG użyto wersji z BatchNorm (VGG_bn) i klasyfikatora $512 \rightarrow 512 \rightarrow C$ z dropoutem (w implementacji: AdaptiveAvgPool2d((1,1)) + nn.Sequential z dwiema warstwami liniowymi i wyjściem C).

VGG - wariant E (VGG-19, „3×3 everywhere”)

Układ zgodny z [17], dostosowany do 32×32 (CIFAR). W plikach **VGG** i **CyVGG** konfiguracja **E** to: [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 512, 'M']. Warstwy budowane są przez fabrykę make_layers(cfg['E'], ...), z opcją BatchNorm (*_bn) i MaxPool2d po symbolu 'M'. Za częścią splotową: AdaptiveAvgPool2d((1,1)), flatten i dwie warstwy liniowe $512 \rightarrow 512 \rightarrow C$ z dropoutem.

- **Blok 1:** 64, 64 $\rightarrow$ MaxPool (32 $\rightarrow$ 16)
- **Blok 2:** 128, 128 $\rightarrow$ MaxPool (16 $\rightarrow$ 8)
- **Blok 3:** $256 \times 4 \rightarrow$ MaxPool (8 $\rightarrow$ 4)
- **Blok 4:** $512 \times 4 \rightarrow$ MaxPool (4 $\rightarrow$ 2)
- **Blok 5:** $512 \times 4 \rightarrow$ MaxPool (2 $\rightarrow$ 1)

W **CyVGG** każda Conv2d jest zastąpiona **CyConv2d** (interfejs jak w Conv2d: 3×3 , padding=1, wsparcie stride/dylacji). Klasyfikator i układ bloków pozostają takie same jak w VGG.

ResNet-56 (CIFAR, $6n+2$ z $n=9$)

Schemat jak w [18] dla CIFAR. Startowa Conv 3×3 , następnie **3 grupy** po **$n=9$** bloków BasicBlock. Grupa 1 (16 kanałów, stride 1), Grupa 2 (32 kanały, pierwszy blok stride 2), Grupa 3 (64 kanały, pierwszy blok stride 2). BasicBlock: Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv $3 \times 3 \rightarrow$ BN + skrót. Downsample - **opcja A** (CIFAR): podpróbkiwanie przestrzenne $x[:, :, ::2, ::2]$ i dopełnienie kanałów F.pad(...). Zakończenie: **GAP** $\rightarrow$ warstwa liniowa $64 \rightarrow C$.

W **CyResNet** zastąpiono wszystkie nn.Conv2d przez **CyConv2d** (w tym conv1 oraz konwolucje w BasicBlock). Reszta (BN, ReLU, shortcut, GAP, Linear) pozostaje bez zmian względem bazowej wersji.

Wersje cykliczne: CyVGG-E i CyResNet-56

- **Conv** → **CyConv**. Każdą Conv2d zastąpiono **CyConv2d**. Interfejs (rozmiary jąder, stride, padding) jest drop-in zgodny z Conv2d, więc topologia i klasyfikator są identyczne jak w bazach.
- **Implementacja warstwy**. CyConv2d opakowuje własną funkcję autograd (CyConv2dFunction) i wywołuje rozszerzenie CUDA CyConv2d_cuda.forward/backward(...). Moduł posiada duży bufor roboczy na GPU (opisany w kodzie jako „Workspace for Cy-Winograd algorithm”). Wagi inicjalizowane są przez xavier_uniform_.
- **Uwaga dot. osi orientacji**. W kodzie modeli **nie ma jawnej dodatkowej osi „orientacja”** ani osobnego „poolingu po orientacjach”. Z poziomu PyTorch interfejs filtrów ma kształt [C_out, C_in, k, k] (jak w standardowym Conv2d). Mechanizmy rotacyjne - jeśli obecne - są enkapsulowane w jądrze CUDA CyConv2d_cuda, niewidocznym w plikach modeli.
- **Inwariancja w praktyce**. Po stronie modeli **GAP** oraz (opcjonalnie) dalsze uśrednianie w klasyfikatorze są identyczne jak w bazach; brak osobnego „poolingu po orientacjach” widocznego w kodzie modeli.

Uzgodnienia I/O i selektor modeli

- **Kanały wejścia**. 1 kanał dla mnist, mnist-custom, GTSRB-custom, LEGO; 3 kanały w pozostałych przypadkach (np. CIFAR, pełny GTSRB).
- **Liczba klas**. MNIST/CIFAR-10: 10; GTSRB: 43; LEGO: 50; CIFAR-100: 100 - zgodnie z helperami get_num_classes.
- **Selektor modeli**. get_model(model, dataset, classify=True) zwraca jedną z wersji: vgg*, cyvgg*, resnet*, cyresnet*, zależnie od stringa modelu i nazwy zbioru danych.

Standardowe CNN

Jako bazy zastosowano **VGG** (wariant **E** / **VGG-19**) i **ResNet-56**. Konwolucje 3×3 z padding=1, w VGG okresowy MaxPool2d(2), w ResNet zmniejszanie rozdzielczości przez stride=2. Po części konwolucyjnej: **GAP** i **klasyfikator** (w VGG: dwie warstwy w pełni połączone 512→512→C z dropoutem; w ResNet: Linear 64→C). Warianty VGG w kodzie występują także w wersjach *_bn (z BatchNorm). Modele te są z natury **ekwawariantne translacyjnie** (dobrze znoszą przesunięcia), lecz nie posiadają wbudowanego mechanizmu inwariancji na rotacje - stanowią więc punkt odniesienia dla wersji cyklicznych [17], [18].

Rotacyjnie inwariantne sieci (CyResNet, CyVGG)

Wersje cykliczne (**CyVGG-E**, **CyResNet-56**) powstały przez **podmianę każdej Conv2d na CyConv2d**. Architektura bloków, liczby kanałów, GAP i układ klasyfikatora pozostają bez zmian, co pozwala porównać wpływ samej warstwy konwolucyjnej. W kodzie modeli **nie ma jawnego wymiaru orientacji** ani dedykowanego „poolingu po orientacjach”; funkcjonalność rotacyjna jest schowana w implementacji CUDA CyConv2d_cuda, wywoływanej z poziomu CyConv2d [4].

Transformacje polarne: linearpolar vs logpolar

Implementacja i środowisko eksperymentalne

Szczegóły implementacyjne: warstwa CyConv2d (CUDA)

Rozszerzenie. Warstwa korzysta z modułu C++/CUDA budowanego jako CyConv2d_cuda z plików cycnn.cpp i cycnn_cuda.cu (setup przez setuptools / BuildExtension).

Interfejs (C++ binding). W cycnn.cpp eksportowane są funkcje forward(...) i backward(...) (pybind11) przyjmujące input, weight, workspace oraz stride, padding, dilation. Makra sprawdzają, że tensory są **CUDA** i **contiguous**, po czym wywoływane są implementacje cyconv2d_cuda_forward/backward.

Warstwa w PyTorch. CyConv2dFunction (autograd) wywołuje CyConv2d_cuda.forward/backward i przekazuje **workspace** oraz parametry geometrii. Moduł CyConv2d trzyma wagi o kształcie [C_out, C_in, k, k] (inicjowane xavier_uniform_) i w forward używa CyConv2dFunction.apply(...).

Bufor roboczy. CyConv2d.workspace to prealokowany tensor float32 na GPU o rozmiarze 1024*1024*1024 elementów (~4 GiB), opisany jako „Workspace for Cy-Winograd algorithm”. To może powodować **OOM** na kartach z mniejszą pamięcią VRAM.

Integracja z modelami. Wszystkie nn.Conv2d w wariantach CyVGG i CyResNet są zastąpione CyConv2d (m.in. conv1 oraz konwolucje w blokach). Topologia, BN/ReLU, GAP i klasyfikator są zachowane 1:1 względem bazowych wersji.

Uwaga merytoryczna. W kodzie modeli nie widać jawnej osi „orientacji” ani osobnego **poolingu po orientacjach**; z poziomu PyTorch wagi mają klasyczny kształt [C_out, C_in, k, k]. Jeśli własności rotacyjne występują, to są **enkapsulowane w jądrze CUDA** (cycnn_cuda.cu) wywoływanym przez bindingi.

Python, PyTorch

Struktura projektu

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Obsługa GPU, Docker, WSL

Organizacja logów, modeli, confusion matrixów

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi - nie można zjeść ciastka i mieć go też (ang. „You can’t eat your cake and have it too” [41]).

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników

Bibliografia

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: <https://www.deeplearningbook.org/>
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning.” 2016. Available: <https://arxiv.org/abs/1603.07285>
- [3] T. Cohen and M. Welling, “Group equivariant convolutional networks,” in *Proceedings of the 33rd international conference on machine learning (ICML)*, 2016. Available: <https://proceedings.mlr.press/v48/cohen16.html>
- [4] J. Kim, W. Jung, H. Kim, and J. Lee, “CyCNN: A rotation invariant CNN using polar mapping and cylindrical convolution layers,” *arXiv preprint arXiv:2007.10588*, 2020, Available: <https://arxiv.org/abs/2007.10588>
- [5] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems 32 (NeurIPS)*, 2019. Available: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [6] Python Software Foundation, “Python 3 documentation.” 2025. Available: <https://docs.python.org/3/>
- [7] PyTorch Core Team, “PyTorch documentation (stable).” 2025. Available: <https://pytorch.org/docs/stable/>
- [8] NVIDIA Corporation, “CUDA toolkit documentation.” 2025. Available: <https://docs.nvidia.com/cuda/>
- [9] NVIDIA Corporation, “NVIDIA cuDNN developer guide.” 2025. Available: <https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html>
- [10] “NVIDIA tensor cores.” 2024. Available: <https://developer.nvidia.com/tensor-cores>
- [11] “NVIDIA ampere architecture: TF32 tensor cores.” 2024. Available: <https://developer.nvidia.com/blog/tf32-matmul>
- [12] P. Micikevicius *et al.*, “Mixed precision training,” in *International conference on learning representations (ICLR)*, 2018.
- [13] “Automatic mixed precision (AMP) - PyTorch docs.” 2024. Available: <https://pytorch.org/docs/stable/amp.html>
- [14] “Ubuntu documentation.” 2024. Available: <https://help.ubuntu.com/>
- [15] “Windows subsystem for linux (WSL) documentation.” 2024. Available: <https://learn.microsoft.com/windows/wsl/>
- [16] Docker, Inc., “Docker documentation.” 2025. Available: <https://docs.docker.com/>
- [17] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014, Available: <https://arxiv.org/abs/1409.1556>
- [18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 770–778. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

- [19] B. S. Reddy and B. N. Chatterji, “An FFT-based technique for translation, rotation and scale-invariant image registration,” *IEEE Transactions on Image Processing*, vol. 5, no. 8, pp. 1266–1271, 1996, doi: 10.1109/83.506761.
- [20] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” 1998. Available: <http://yann.lecun.com/exdb/mnist/>
- [21] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The german traffic sign recognition benchmark: A multi-class classification competition,” in *Proceedings of the international joint conference on neural networks (IJCNN)*, 2011, pp. 1453–1460. doi: 10.1109/IJCNN.2011.6033395.
- [22] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-CNN,” in *Proceedings of the IEEE international conference on computer vision (ICCV)*, 2017. Available: https://openaccess.thecvf.com/content_ICCV_2017/papers/He_Mask_R-CNN_ICCV_2017_paper.pdf
- [23] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, in LNCS, vol. 9351. 2015, pp. 234–241. Available: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28
- [24] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, Available: <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>
- [25] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu, “Spatial transformer networks,” in *Advances in neural information processing systems (NIPS)*, 2015. Available: <https://proceedings.neurips.cc/paper/2015/file/33ceb07bf4eeb3da587e268d663aba1a-Paper.pdf>
- [26] C. Esteves, C. Allen-Blanchette, A. Makadia, and K. Daniilidis, “Learning SO(3)-equivariant representations with spherical CNNs,” in *Proceedings of the european conference on computer vision (ECCV)*, 2018, pp. 52–68. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Carlos_Esteves_Learning_SO3_Equivariant_ECCV_2018_paper.html
- [27] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*, 2nd ed. Cambridge University Press, 2004.
- [28] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proceedings of the 37th international conference on machine learning (ICML)*, PMLR, 2020. Available: <https://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
- [29] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020. Available: https://openaccess.thecvf.com/content_CVPR_2020/papers/He_Momentum_Contrast_for_Unsupervised_Visual_Representation_Learning_CVPR_2020_paper.pdf
- [30] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: Bandit-based configuration evaluation for hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Available: <https://jmlr.org/papers/volume18/16-558/16-558.pdf>

- [31] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *International conference on learning representations (ICLR)*, 2019. Available: <https://web.engr.oregonstate.edu/~tgd/publications/hendrycks-dietterich-benchmarking-neural-network-robustness-to-common-corruptions-and-perturbations-iclr2019.pdf>
- [32] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout.” 2017. Available: <https://arxiv.org/abs/1708.04552>
- [33] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [34] A. Azulay and Y. Weiss, “Why do deep convolutional networks generalize so poorly to small image transformations?” in *Journal of vision*, 2019, pp. 1–14.
- [35] W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks,” in *Advances in neural information processing systems (NeurIPS)*, 2016.
- [36] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International conference on machine learning (ICML)*, 2015.
- [37] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International conference on learning representations (ICLR)*, 2014.
- [38] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, “Geometric deep learning: Grids, groups, graphs, geodesics, and gauges,” *arXiv preprint arXiv:2104.13478*, 2021, Available: <https://arxiv.org/abs/2104.13478>
- [39] M. Weiler and G. Cesa, “General $e(2)$ -equivariant steerable CNNs,” *arXiv preprint arXiv:1911.08251*, 2019, Available: <https://arxiv.org/abs/1911.08251>
- [40] T. S. Cohen, M. Geiger, and M. Weiler, “A general theory of equivariant CNNs on homogeneous spaces,” in *Advances in neural information processing systems (NeurIPS)*, 2019. Available: <https://papers.neurips.cc/paper/9114-a-general-theory-of-equivariant-cnns-on-homogeneous-spaces.pdf>
- [41] T. J. Kaczynski, “Industrial society and its future,” *The Washington Post*, Sep. 1995, Available: <https://www.washingtonpost.com/wp-srv/national/longterm/unabomber/manifesto.text.htm>